

Université Assane Seck de Ziguinchor
UFR des Sciences et Technologies
Département d'Informatique

Année universitaire 2023-2024

Master 1 en Informatique, option GL
Unité d'Enseignement : Systèmes à base de connaissances
Élément Constitutif : Programmation fonctionnelle

Responsable CM / TD : Mouhamadou GAYE

TD 3

Exercice 1

Soit le type de module suivant:

```
module type Fract =  
  sig  
    type t = int*int  
    exception FractionMalDefinie  
    val simplifie : t -> t  
  
  end;;
```

Définir le module Fraction ayant cette signature. La fonction simplifie doit rendre la forme irréductible de la fraction.

Exercice 2

On donne la signature d'un module Urne permettant de faire un vote à bulletin secret :

- Il y a deux candidats A et B;
- Chaque électeur est doté d'un numéro entier (unique), ce qui permet de s'assurer qu'un même électeur ne peut pas comptabiliser deux votes;
- En cours de scrutin, un électeur peut changer d'avis et revoter (l'ancien vote est alors détruit);
- On peut dépouiller l'urne pour connaître les scores de chaque candidat et déterminer le vainqueur.

Le type t est celui de l'urne proprement dite.

```

module type URNE =
  sig

    type electeur = int
    type candidat = A | B
    type t (* type de l'urne *)
    val vide : t
    val voter : electeur → candidat → t → t
    val depouiller : t → int × int
    val vainqueur : t → candidat

  end;;

```

Exercice 3

On veut définir des structures d'arbres paramétrés par le type 'a des informations contenues dans les nœuds. Les informations des nœuds seront organisées comme dans un arbre de recherche, à savoir, pour chaque nœud dont l'information est i, toutes les informations dans le sous arbre gauche sont plus petites ou égales à i et toutes celles dans le sous arbre droite sont plus grandes que i. Les arbres construits seront impératifs, c.à.d., l'ajout d'une branche dans un arbre sera implémenté par la modification de celle-ci. Ainsi, les branches gauches et droite d'un nœud seront définies mutables. Les méthodes qu'on souhaite avoir sont :

- **is_void : bool** qui renvoie true si l'arbre est vide et false sinon;
- **insert : 'a -> unit** qui rajoute un élément à l'arbre selon la discipline des arbres binaires de recherche (on utilisera la relation polymorphe <);
- **get_left** et **get_right** qui donnent respectivement les fils gauche et droit de l'arbre;
- **to_list : 'a list** qui rend la liste des éléments contenus dans l'arbre (de la gauche vers la droite, et du plus profond vers le haut).

On utilisera deux classes pour modéliser les deux cas possibles d'arbre

- **class ['a] nœud info a1 a2:** modélise l'arbre non vide, qui contient l'information info et les sous arbres a1 et a2;
 - **class ['a] void:** modélise l'arbre vide (sans sous arbres ni information).
1. Quels sont les types des méthodes **get_left** et **get_right** ?
 2. Définir la classe **['a] node** pour représenter les arbres binaires non vides. L'interface de la classe doit être:

```

class ['a] node : 'a -> 'a node -> 'a node ->

```

object

```
val label : 'a
val mutable left : 'a node
val mutable right : 'a node
method insert : 'a -> unit
method get_left : 'a node
method get_right : 'a node
method is_void : bool
method to_list : 'a list
```

end

3. Définir la classe [`'a`] `void` pour représenter l'arbre vide. Les méthodes `insert`, `get_left` et `get_right` échoueront toujours. L'interface de la classe doit être :

```
class ['a] void :
```

```
object ('b)
```

```
method insert : 'a -> unit
method get_left : 'b
method get_right : 'b
method is_void : bool
method to_list : 'a list
```

end

Noter ici que le types des objets de cette classe est (`'b`) (autrement dit, `'b = 'a void`). C'est en particulier le type renvoyé par les deux méthodes `get_left` et `get_right`. Ce typage est crucial pour la compatibilité qui sera nécessaire afin de typer un sous arbre qui peut être vide (donc, de type `'a void`) ou un nœud (donc, de type `'a noeud`).

4. Construire un objet contenant contenant l'arbre suivant :

